

บทที่ 10

เทคนิคในการพัฒนาเกม 3 มิติ

10.1 เอนิเมชัน (Animation)

เอนิเมชันหรือการทำภาพเคลื่อนไหวเป็นเทคนิคที่มีความซับซ้อนมากยิ่งขึ้น ซึ่งเทคนิคที่นิยมใช้กันมี 2 ประเภทคือ เทคนิคคีย์เฟรมอินเทอร์โพลชัน (Keyframe Interpolation Techniques) และเทคนิคสกินนิ่ง (Skinning Techniques)

10.1.1 การทำเอนิเมชันโดยใช้เทคนิคคีย์เฟรมอินเทอร์โพลชัน (Keyframe Interpolation Technique)

คีย์เฟรม (Keyframe) คือ การสร้างการกระทำจากคีย์ด้วยการกำหนดคุณสมบัติให้แตกต่างกันไปตามเวลาที่เปลี่ยนแปลงไป

การใช้เทคนิคของคีย์เฟรมอินเทอร์โพลชันใน Direct3D มีหลักการดังนี้

- เก็บตำแหน่งของเวอร์เท็กซ์ไว้เป็นชุดตามค่าคีย์เฟรม การทำเอนิเมชันสามารถทำได้โดยการเฉลี่ยค่าระหว่างตำแหน่งของเวอร์เท็กซ์ในแต่ละชุดโดยใช้ค่าของเวลาเป็นตัวกำหนดน้ำหนักของการเฉลี่ย (Blending)
- สามารถเฉลี่ยค่าคุณสมบัติอื่นๆ ได้ด้วยวิธีการเดียวกัน เช่น ค่าแกนตั้งฉาก (normal) หรือค่าคู่อันดับกำหนดตำแหน่งของเทกเจอร์ (texture coordinate)
- การเฉลี่ยค่าสามารถทำได้โดยฮาร์ดแวร์ได้ (ทำบน GPU) ด้วยการใช้เวอร์เท็กซ์เชดเดอร์ (vertex shader) และสตรีมข้อมูลเวอร์เท็กซ์หลายๆ สตรีม

ตัวอย่าง ผลงานที่ได้เกิดจากการใช้เทคนิคคีย์เฟรมอินเทอร์โพลชัน



รูปที่ 10-1 รูปแสดงภาพเคลื่อนไหวที่เกิดจากเทคนิคคีย์เฟรมอินเทอร์โพลชัน

การใช้เทคนิคคีย์เฟรมอินเทอร์โพลชันมี 2 ชนิด แบ่งตามวิธีการคำนวณค่าเวอร์เท็กซ์ คือ

10.1.1.1 เทคนิคคีย์เฟรมอินเทอร์โพลชันเชิงเส้น (Linear Keyframe Interpolation)

สมการที่ใช้ในการคำนวณค่าเวอร์เท็กซ์คือ

$$v_{final} = v_0 \times (1 - t) + v_1 \times t$$

เมื่อ ค่า V_{final} คือ ค่าที่ได้จากการเฉลี่ยค่า V_0 และค่า V_1 โดยใช้เวลากำหนดน้ำหนักในการเฉลี่ยค่า

10.1.1.2 เทคนิคคีย์เฟรมอินเทอร์โพลชันด้วย Hermite Spline (Hermite Spline Keyframe Interpolation)

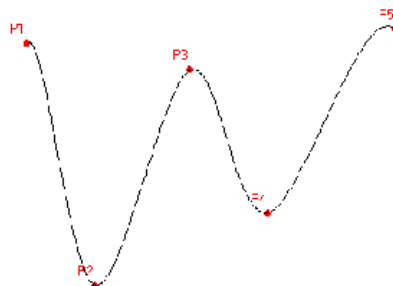
การใช้ Hermite Spline ช่วยให้การเคลื่อนที่ของเวอร์เท็กซ์มีความราบรื่นยิ่งขึ้น เนื่องจาก Spline เป็นสมการที่ใช้แสดงค่าของเส้นโค้ง แต่จะทำให้ต้องประมวลผลข้อมูลเพิ่มขึ้น

สมการที่ใช้ในการคำนวณค่าเวอร์เท็กซ์คือ

$$v_{final} = h_0 \times v_i + h_1 \times (v_{i+1} - v_{i-1}) + h_2 \times (v_{i+2} - v_i) + h_3 \times v_{i+1}$$

การใช้ Hermite Spline Keyframe Interpolation ต้องใช้สตริ์มของข้อมูล 4 สตริ์ม สำหรับเก็บค่าช่วงก่อนคำนวณ 2 สตริ์ม และหลังจากคำนวณ 2 สตริ์ม

รูปแสดงตัวอย่างของ Spline



รูปที่ 10-2 รูปแสดงเส้นโค้ง Spline

ข้อดีของคีย์เฟรมอินเทอร์โพลชัน (Keyframe Interpolation)

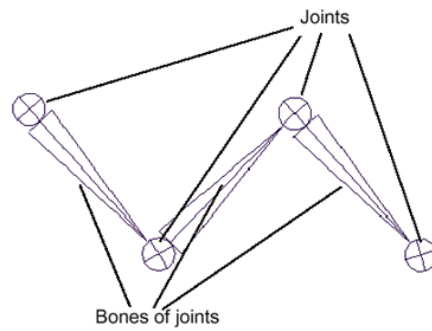
1. สามารถสร้างแอนิเมชันของเวอร์เท็กซ์ในรูปแบบใดๆ ก็ได้ โดยการกำหนดตำแหน่งของเวอร์เท็กซ์ในแต่ละเฟรม
2. การคำนวณในแบบเชิงเส้นเป็นการคำนวณง่ายๆ ที่ไม่มีความซับซ้อน

ข้อเสียของคีย์เฟรมอินเทอร์โพลชัน

1. ต้องใช้ข้อมูลจำนวนมาก เช่น ถ้าโมเดลของเราประกอบด้วยเวอร์เท็กซ์ 2000 เวอร์เท็กซ์ และในการทำท่าทาง 1 ท่า ใช้คีย์เฟรมจำนวน 10 คีย์เฟรม จะต้องเก็บข้อมูลพร้อมๆ กันทีเดียวถึง 2000×10 เท่ากับ 20000 เวอร์เท็กซ์ จึงเหมาะสำหรับวัตถุที่มีจำนวนเวอร์เท็กซ์น้อยๆ เท่านั้น

10.1.2 การทำแอนิเมชันโดยใช้เทคนิคสกินนิง (Skinning Technique)

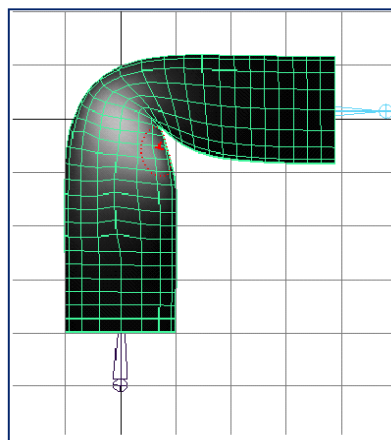
โบน (Bone) และจอยท์ (Joint) เป็นพื้นฐานสำคัญในการใช้เทคนิคสกินนิง จอยท์คือ จุดที่มีการเชื่อมต่อกันกันของโบน ส่วนโบน คือ ความสัมพันธ์ระหว่างจอยท์ 2 จอยท์ ดังรูป



รูปที่ 10-3 รูปแสดงโบนและจอยท์

หลักการของเทคนิคสกินนิ่ง

เทคนิคสกินนิ่ง คือ การกำหนดค่าน้ำหนักให้กับแต่ละเวอร์เท็กซ์ให้มีการเคลื่อนที่สอดคล้องกับโบนที่เคลื่อนที่ไป
ดั่งภาพ

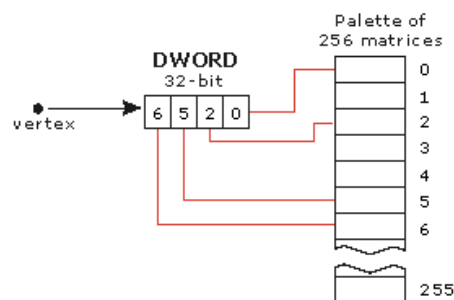


รูปที่ 10-4 รูปแสดงการเฉลี่ยค่าที่เกิดจากการใช้เทคนิคสกินนิ่ง

ใน Direct3D เวอร์ชัน 8 มีวิธีการที่เราจะใช้เทคนิคสกินนิ่ง 2 วิธี คือ

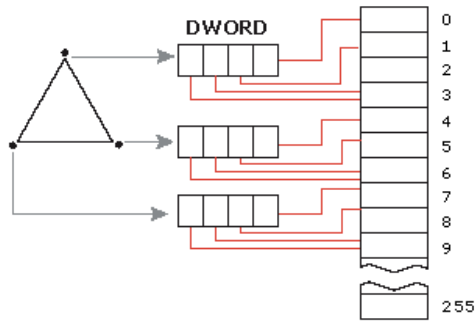
10.1.2.1 Fixed-function matrix palette skinning

หรือ Indexed Vertex Blending ในไคลเร็กทรีดีมีการกำหนดค่าของโบนในรูปของเมตริกซ์ โดยการอ้างอิงโบนจะใช้
ค่าอินเด็กซ์ ซึ่งสามารถกำหนดค่าอินเด็กซ์ (index) ของโบน (bone) ที่จะใช้ได้ถึง 4 ค่าต่อ 1 เวอร์เท็กซ์และพาเลต (palette)
ของโบนสามารถมีได้ถึง 256 เมตริกซ์ ดังรูป



รูปที่ 10-5 รูปแสดงการอ้างอิงข้อมูลในเวอร์เท็กซ์ซึ่งชี้ไปยังเมตริกซ์

หรือ สามารถใช้ได้ถึง 12 ค่าต่อ 1 รูปสามเหลี่ยม ดังรูป



รูปที่ 10-6 รูปแสดงการอ้างอิงข้อมูลจากพื้นผิวไปยังเมตริกซ์

การคำนวณค่าของเวอร์เท็กซ์ สามารถทำได้ดังสมการ

$$V_{world} = V_m \times M[index\ 0] \times b_0 + V_m \times M[index\ 1] \times b_1 + V_m \times M[index\ 2] \times b_2 + V_m \times M[index\ 3] \times (1 - b_0 - b_1 - b_2)$$

V_{model} คือ ค่าตำแหน่งของเวอร์เท็กซ์ใน model space

$Index0..Index3$ คือค่าของอินเด็กซ์ของเมตริกซ์ต่อเวอร์เท็กซ์ 1 ค่าซึ่งรวมกันอยู่ในรูปของ DWORD

$M[]$ คือ อาร์เรย์ของ world เมตริกซ์ที่ใช้ค่าอินเด็กซ์เป็นตัวชี้

$b_0..b_2$ คือ ค่าน้ำหนักในการเฉลี่ยสำหรับเมตริกซ์ในแต่ละค่าอินเด็กซ์

V_{world} คือ ผลลัพธ์ที่ได้ซึ่งอยู่ใน world space

การกำหนดค่าของ Indexed Vertex Blending Transform และสถานะของการเรนเดอร์

สามารถกำหนดค่าของเมตริกซ์ได้ด้วยการเรียกเมทอด `IDirect3DDevice8::SetTransform` แล้วใช้มาโคร `D3DTS_WORLDMATRIX` ในการกำหนดค่าให้กับเมตริกซ์โดยใช้ค่าอินเด็กซ์ระหว่าง 0-255 ดังตัวอย่าง

```
D3DMATRIX matBlend1;
```

```
D3DXMatrixIdentity( &matBlend1 );
```

```
d3dDevice->SetTransform( D3DTS_WORLDMATRIX(0), &matBlend );
```

ในการใช้งาน Indexed Vertex Blending ยังต้องกำหนดสถานะการเรนเดอร์อีกด้วยโดยการใช้เมทอด `IDirect3DDevice8::SetRenderState` ใส่ค่า `D3DRS_INDEXEDVERTEXBLENDENABLE` ในพารามิเตอร์ตัวแรก และค่า `TRUE` ในพารามิเตอร์ตัวที่สอง ดังตัวอย่าง

```
d3dDevice->SetRenderState( D3DRS_INDEXEDVERTEXBLENDENABLE, TRUE );
```

นอกจากนั้น ยังต้องกำหนดจำนวนของอินเด็กซ์ที่จะใช้ในการคำนวณด้วยการเรียกเมทอด

IDirect3DDevice::SetRenderState ใส่ค่า D3DRS_VERTEXBLEND เป็นพารามิเตอร์แรก และใส่ค่าซึ่งเป็นสมาชิกของชนิดข้อมูล D3DVERTEXBLEND_FLAGS enumerated type เป็นพารามิเตอร์ตัวที่สอง ดังตัวอย่าง

```
d3dDevice->SetRenderState( D3DRS_VERTEXBLEND, D3DVBF_3WEIGHTS );
```

ข้อดีของการใช้ Fixed-function matrix palette skinning

2. ใช้งานง่าย
3. ใช้ข้อมูลจำนวนน้อยในการทำอนิเมชัน 1 ท่า เนื่องจากเก็บเฉพาะค่าเมตริกซ์ที่เป็นคีย์เท่านั้น
4. สามารถเพิ่มเติมข้อมูลอนิเมชันเข้าไปใหม่ได้โดยไม่ต้องแก้ไขโมเดล

ข้อเสีย

1. ไม่มีฮาร์ดแวร์ที่ช่วยเร่งความเร็วในการทำงาน และฮาร์ดแวร์ที่มีอยู่ก็ไม่สามารถใช้ได้ถึง 256 โบน
2. ซอฟต์แวร์ที่มีอยู่ใน Direct3D เวอร์ชัน 8 ยังไม่มีประสิทธิภาพในการทำงานดีพอ
3. ไม่สามารถทำงานกับ per-pixel lighting ได้ดี
4. จำกัดจำนวนโบนต่อเวอร์เท็กซ์ไม่เกิน 4 โบน

10.1.2.2 การเขียนโปรแกรมเวอร์เท็กซ์เชดเดอร์ (Vertex Shader)

ข้อดี

1. มีฮาร์ดแวร์ที่สนับสนุนการทำงาน เช่น GeForce3
2. ซอฟต์แวร์ที่มีประสิทธิภาพสูงกว่า
3. ไม่มีการจำกัดจำนวนโบนต่อเวอร์เท็กซ์
4. ทำงานได้อย่างดีกับ per-pixel lighting

ข้อเสีย

1. มีความซับซ้อนสูงมาก
2. มีความจำเป็นที่จะต้องแบ่งกลุ่มของไพรมีทีฟตามจำนวนโบน

ตัวอย่าง โค้ดในการทำสกินนิ่งจำนวน 2 โบนต่อ 1 เวอร์เท็กซ์ด้วยเวอร์เท็กซ์เชดเดอร์จะเห็นว่ามีความซับซ้อนมากกว่าวิธี Indexed Vertex Blending มาก

```
//load first index
```

```
mov a0.x, V_INDICES.x
```

```
//transform position by first bone, store in r0
```

```
dp4 r0.x, V_POSITION, c[a0.x + CV_BONESTART + 0]
```

```
dp4 r0.y, V_POSITION, c[a0.x + CV_BONESTART + 1]
```

```
dp4 r0.z, V_POSITION, c[a0.x + CV_BONESTART + 2]
```

```
//transform normal by first bone, store in r2
```

```

dp3 r2.x, V_NORMAL, c[a0.x + CV_BONESTART + 0]
dp3 r2.y, V_NORMAL, c[a0.x + CV_BONESTART + 1]
dp3 r2.z, V_NORMAL, c[a0.x + CV_BONESTART + 2]
//load second index
mov a0.x, V_INDICES.y
//transform position by second bone, store in r1
dp4 r1.x, V_POSITION, c[a0.x + CV_BONESTART + 0]
dp4 r1.y, V_POSITION, c[a0.x + CV_BONESTART + 1]
dp4 r1.z, V_POSITION, c[a0.x + CV_BONESTART + 2]
//transform normal by second bone, store in r3
dp3 r3.x, V_NORMAL, c[a0.x + CV_BONESTART + 0]
dp3 r3.y, V_NORMAL, c[a0.x + CV_BONESTART + 1]
dp3 r3.z, V_NORMAL, c[a0.x + CV_BONESTART + 2]
//blend between r0 and r1 -- the positions
mul r0, r0, V_WEIGHT.x
mad r1, r1, V_WEIGHT.y, r0
//blend between r2 and r3 -- the normals
mul r2, r2, V_WEIGHT.x
mad r3, r3, V_WEIGHT.y, r7

```

10.2 การตรวจสอบการชนของวัตถุ (Collision Detection)

10.2.1 วัตถุตรวจสอบการชนพื้นฐาน

ในโครงงานนี้การตรวจสอบการชนของวัตถุจะใช้การตรวจสอบกับวัตถุการชนพื้นฐาน 4 แบบ ดังนี้

1. แบบพื้น (Ground type)
2. แบบกำแพง (Wall type)
3. แบบทรงกระบอก (Cylinder type)
4. แบบทรงกลม (Sphere type)

10.2.2 การตรวจสอบการชนกับวัตถุแบบพื้น

โดยปกติแล้วตัวละครที่ปรากฏอยู่ในเกมมักจะเดินอยู่บนพื้นตามแรงดึงดูดของโลก ซึ่งเราสามารถจำลองคุณสมบัตินี้ได้โดยสร้างข้อมูลที่ชี้แทนพื้นขึ้นมา ซึ่งตัวละครในเกมจะต้องเลื่อนหรือไถลไปตามพื้นผิวของวัตถุตรวจสอบการชนนี้

ข้อมูลที่ใช้ในสร้างวัตถุตรวจสอบการชนแบบพื้น

- จุด 3 จุดซึ่งประกอบกันเป็นรูปสามเหลี่ยม

ข้อมูลที่มีอยู่ในวัตถุตรวจสอบการชนแบบพื้น

- สมการเส้นตรงซึ่งแสดงขอบเขตของรูปสามเหลี่ยม

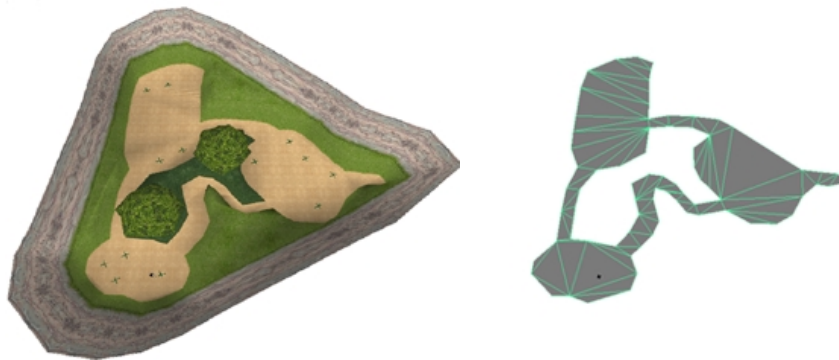
- ค่า Boolean ซึ่งแสดงว่าผลลัพธ์ที่ได้จากการแทนค่าในสมการเส้นตรงแบบใดคือ จุดที่อยู่ในรูปสามเหลี่ยม
- สมการระนาบของรูปสามเหลี่ยมนั้น
- เมตริกซ์อ้างอิง

ข้อมูลที่ถูกใช้ในการตรวจสอบ

- ค่าตำแหน่งของวัตถุ

ผลลัพธ์ที่ได้จากการตรวจสอบการชนแบบพื้น

- ความสูงในแนวตั้งฉากกับรูปสามเหลี่ยมที่พบว่าตำแหน่งวัตถุอยู่ภายใน
- สมการระนาบของรูปสามเหลี่ยมนั้น



รูปที่ 10-7 รูปแสดงวัตถุตรวจสอบการชนแบบพื้นที่ได้จากฉาก

10.2.3 การตรวจสอบการชนกับวัตถุแบบกำแพง

ตรงข้ามกับวัตถุแบบพื้น วัตถุแบบกำแพงจะวางตัวในแนวแกนที่เราสนใจ ซึ่งโดยปกติคือ แกน y นั่นคือ วัตถุชนิดนี้จะเป็นตัวขวางกั้นคล้ายกับกำแพงในชีวิตประจำวัน

ข้อมูลที่ใช้ในสร้างวัตถุตรวจสอบการชนแบบพื้น

- จุด 3 จุดซึ่งประกอบกันเป็นรูปสามเหลี่ยม

ข้อมูลที่มีอยู่ในวัตถุตรวจสอบการชนแบบพื้น

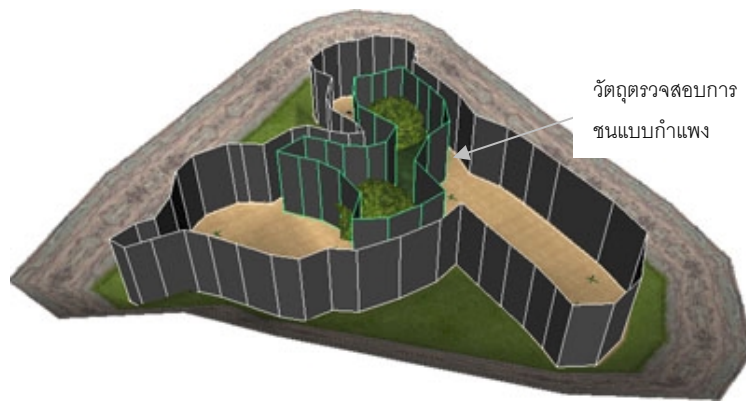
- ตำแหน่งอ้างอิง
- ความสูง
- สมการเส้นตรง 1 สมการ ซึ่งแทนเส้นตรงที่เกิดจากกำแพงเมื่อมองลงมาจากแกน y
- เมตริกซ์อ้างอิง

ข้อมูลที่ถูกใช้ในการตรวจสอบ

- วัตถุตรวจสอบการชนแบบทรงกระบอก

ผลลัพธ์ที่ได้จากการตรวจสอบการชนแบบพื้น

- เวกเตอร์ตั้งฉากกับสมการเส้นตรงที่ตัดผ่านจุดศูนย์กลางของทรงกระบอก



รูปที่ 10-8 รูปแสดงวัดตรวสอบการชนแบบกำแพงที่ได้จากฉาก

10.2.4 การตรวจสอบการชนกับวัตถุแบบทรงกระบอก

วัดตรวสอบการชนแบบทรงกระบอกเป็นตัวกำหนดขอบเขตของตัวละคร วัดตรวสอบนี้ใช้ในการตรวจสอบเพื่อป้องกันการชนกันของตัวละคร และสามารถประยุกต์ใช้งานอื่นๆ ได้อีกมากมาย

ข้อมูลที่ใช้ในสร้างวัดตรวสอบการชนแบบพื้น

- ความสูง, รัศมี
- ตำแหน่งอ้างอิง

ข้อมูลที่มีอยู่ในวัดตรวสอบการชนแบบพื้น

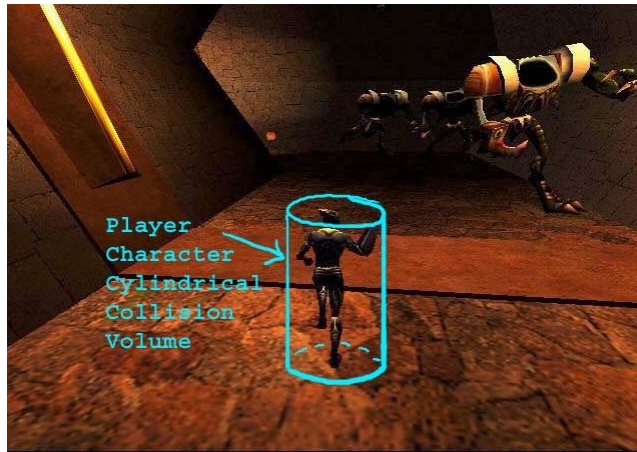
- ตำแหน่งอ้างอิง
- ความสูงและรัศมี
- เมตริกซ์อ้างอิง

ข้อมูลที่ถูกใช้ในการตรวจสอบ

- วัดตรวสอบการชนแบบทรงกระบอก
- วัดตรวสอบการชนแบบกำแพง
- วัดตรวสอบการชนแบบทรงกลม
- ตำแหน่งอ้างอิงใช้ตรวจสอบกับวัดตรวสอบการชนแบบพื้น

ผลลัพธ์ที่ได้จากการตรวจสอบการชนแบบพื้น

- เวกเตอร์ตั้งฉากซึ่งตัดผ่านจุดศูนย์กลางของวัดตรวสอบการชนแบบทรงกระบอกและวัดตรวสอบการชนแบบทรงกลม



รูปที่ 10-9 รูปแสดงวัตถุตรวจสอบการชนแบบทรงกระบอกรอบตัวละคร

10.2.5 การตรวจสอบการชนกับวัตถุแบบทรงกลม

ในกรณีที่ต้องการการตรวจสอบที่มีความละเอียดยิ่งขึ้น วัตถุตรวจสอบการชนแบบทรงกลมจะตอบสนองความต้องการในส่วนนี้ได้ เนื่องจากการใช้วัตถุตรวจสอบการชนชนิดนี้เหมาะสมจะได้ผลลัพธ์ที่มีความใกล้เคียงกับความเป็นจริงที่สุด

ข้อมูลที่ใช้ในสร้างวัตถุตรวจสอบการชนแบบพื้น

- รัศมี
- ตำแหน่งอ้างอิง

ข้อมูลที่มีอยู่ในวัตถุตรวจสอบการชนแบบพื้น

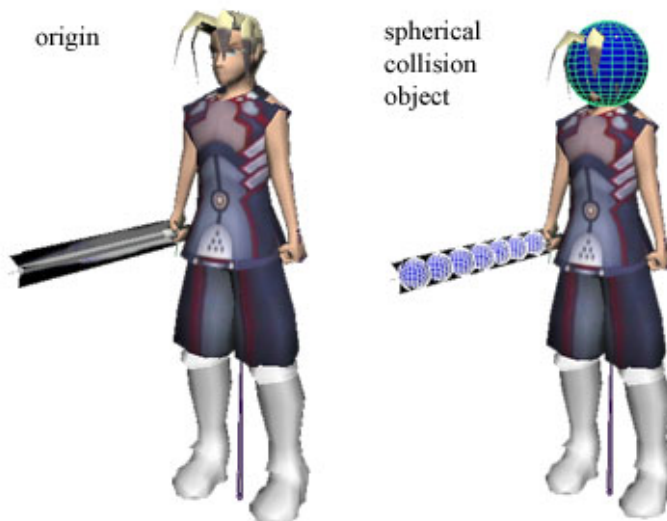
- ตำแหน่งอ้างอิง
- รัศมี
- เมตริกซ์อ้างอิง

ข้อมูลที่ถูกใช้ในการตรวจสอบ

- วัตถุตรวจสอบการชนแบบทรงกระบอก
- วัตถุตรวจสอบการชนแบบก้ำแกง
- วัตถุตรวจสอบการชนแบบทรงกลม

ผลลัพธ์ที่ได้จากการตรวจสอบการชนแบบพื้น

- เวกเตอร์ตั้งฉากซึ่งตัดผ่านจุดศูนย์กลางของวัตถุตรวจสอบการชนแบบทรงกระบอกและวัตถุตรวจสอบการชนแบบทรงกลม
- เวกเตอร์ซึ่งตั้งฉากกับวัตถุตรวจสอบการชนแบบก้ำแกงซึ่งตัดผ่านจุดศูนย์กลางของวัตถุตรวจสอบการชนแบบทรงกลม



รูปที่ 10-10 รูปแสดงวัตถุตรวจสอบการชนแบบทรงกลมที่สร้างขึ้นในส่วนที่ต้องการบนตัวละคร

10.3 Light Map (Dark Map)

เป็นวิธีการที่นิยมใช้ในการเพิ่มความสมจริงให้กับฉากในเรื่องของแสงและเงา เนื่องจากการประมวลผลที่น้อยกว่าและมีวิธีการที่ไม่สลับซับซ้อนมาก เนื่องจากวิธีนี้เป็นการเพิ่มเทกเจอร์เข้าไปอีก 1 ชั้น ซึ่งเทกเจอร์ชั้นที่เพิ่มเข้ามานี้จะเป็นตัวกำหนดความสว่างของวัตถุที่ปรากฏขึ้นมาด้วย ซึ่งการที่จะให้ผลลัพธ์มีความถูกต้องจะต้องกำหนดสถานะชั้นเทกเจอร์ที่เพิ่มเข้ามา ดังตัวอย่าง

```
pDevice->SetTextureStageState( 1, COLORARG1, D3DDTA_TEXTURE);
pDevice->SetTextureStageState( 1, COLORARG2, D3DDTA_CURRENT);
pDevice->SetTextureStageState( 1, COLOROP, D3DTOP_MODULATE);
pDevice->SetTexture(1, pddsLightMap);           // set lightmap texture to be used
```



รูปที่ 10-11 รูปแสดงการใช้ *light map* เพื่อเพิ่มแสงเงาให้กับฉาก